SMOOTH BY DESIGN: CURVE DENOISING WITH AUTOENCODERS

Saeedeh Barzegar Khalilsaraei[1], Jianmin Zheng[2], Ursula Augsdörfer[1]
[1] *Institute of Visual Computing, Graz University of Technology, Graz, Austria*
[2]*College of Computing and Data Science, Nanyang Technological University, Singapore*

ABSTRACT

Curve data collected in practice is often contaminated with noise, which limits its reliability for analysis and visualization in downstream applications. Denoising such curves is therefore a crucial step for ensuring accurate interpretation and broader usability across domains. Autoencoders have proven effective in extracting meaningful representations from noisy data. In this work, we introduce an inception-based autoencoder that transforms an ordered sequence of noisy curve data into a smooth polygonal curve that closely preserves the structure of the original data. Unlike many existing approaches, our method does not impose restrictive assumptions on the global shape of the curve. Instead, it assumes that the data is sampled from an underlying smooth curve or surface. By leveraging this assumption, the model is able to reconstruct not only the geometry but also the differential properties of the original curve. The proposed architecture accommodates varying input sizes and learns robust representations of noisy inputs. Extensive experiments show that our inception-based design achieves superior performance compared to several classical and neural network-based denoising methods.

KEYWORDS

Curve denoising. Deep learning. B-spline curves. Geometry processing.

1. INTRODUCTION

The precise representation of geometric shapes is essential in many applications, yet raw data captured by sensors or derived from manual sketches is often contaminated by noise and fragmentation. In domains such as computer-aided design (CAD), medical imaging, robotics, and geographical information systems, there is a need to reconstruct smooth and geometrically consistent curves from imperfect observations. In practical scenarios, noisy curve data may contain heterogeneous noise distributions, non-uniform sampling densities, outliers, and self-intersections, making robust reconstruction particularly challenging. Beyond geometric accuracy, many downstream applications require preservation of smooth tangential and curvature behavior for reliable analysis, visualization, and further geometric processing.

While classical techniques perform well under simple noise assumptions, they often struggle with heterogeneous noise, irregular sampling, and outliers, and typically require manual parameter tuning (Fan and Pang, 2009). Neural networks offer a data-driven alternative, though many existing approaches assume fixed-size inputs or specific noise models.

In this work, we address the problem of reconstructing smooth polygonal curves from noisy ordered point data using a deep neural network. Rather than explicitly recovering parametric representations, the proposed framework learns spline-like geometric behavior directly from sampled noisy curves. B-spline curves are used to generate training data with consistent smooth geometric behavior (De Boor, 1972). The proposed denoising autoencoder incorporates inception modules (Szegedy et al., 2015) for multi-scale feature extraction and includes a second-order Laplacian regularization term to encourage smooth tangential and curvature behavior in the reconstructed curves. The main contributions of this work are summarized as follows:

- An inception-based denoising autoencoder with a recurrent layer for variable-length curve denoising.
- A smoothness-aware loss combining Laplacian, endpoint, and chord length terms for curve reconstruction.
- Generalization to combined noise types at inference time, despite training only on individual noise types separately.

2. RELATED WORK

Classical curve smoothing techniques include local polynomial fitting such as the *Savitzky–Golay* algorithm (Savitzky and Golay, 1964), locally weighted regression method (Cleveland, 1979), and averaging window filter (Johnston et al., 1999). While effective in simple settings, these approaches are sensitive to parameter choices, often struggle with outliers, and tend to over-smooth. Several geometry-based methods rely on explicit geometric models or optimization schemes for curve reconstruction and denoising (Cheng et al., 2003; Feiszli, 2011; Lu, 2015; Ebrahimi, 2019), which can be computationally expensive and limited in handling outliers. Recent approaches have explored fitting B-spline curves directly to noisy input data. A firefly-based optimization method (Gálvez, 2013) is sensitive to parameter selection. SwarmCurves (Komar, 2023) reconstructs B-spline curves using particle swarm optimization, but it is too slow for real-time applications.

Deep learning approaches for curve reconstruction (Laube et al., 2018; Gao et al., 2019) do not address noisy input data. InceptCurves (Khalilsaraei et al., 2024) introduced an inception-based DNN for B-spline reconstruction, but it is limited to specific noise types. ResCNN (Sun et al., 2020) employs residual convolutional learning for EEG signal denoising and can be adapted to 2D curves. (Morvan et al., 2022) proposed a transformer-based denoising approach using self-attention mechanisms (Vaswani et al., 2017); however, their method targets 1D astronomical light curves rather than 2D geometric curve reconstruction, and transformers can become computationally expensive for densely sampled data.

More recent work targets related but distinct problems: curve reconstruction or parameterization from clean/unordered input (Zou et al., 2025; Lin and Feng, 2025), classical non-learned denoising of unordered 2D point sets (Reghunath et al., 2025), and deep learning denoising of unordered 3D point clouds (Luo and Hu, 2021). Despite this progress, no existing method directly addresses denoising of ordered 2D point sequences while handling variable curve sizes and preserving smooth geometric behavior across diverse noise conditions.

3. APPROACH

We propose an inception-based denoising autoencoder trained on synthetic B-spline curves, combining multi-scale feature extraction with smoothness aware regularization. The following sections describe the dataset, network architecture, and loss function.

3.1 Dataset generation

Synthetic dataset generation enables controlled variation of curve geometry, sampling density, and noise characteristics, which would be difficult to obtain consistently from real-world datasets. We derive a synthetic dataset by sampling B-spline curves, which are widely used for representing smooth and controllable shapes. Using B-spline curves enables the network to learn smooth geometric behavior with continuous derivatives and reconstruct curves with smooth tangents and curvature.

To synthesize training data, random control points ranging from 4 to 10 are generated within the interval [-1,1], from which planar open cubic B-spline curves are derived. To increase curve complexity, control points are reordered so that some curves contain intersections. The curves are then sampled uniformly or non-uniformly using between 20 and 500 sample points, which serve as the target curves for evaluating the loss function. To generate the network input, the sampled points are corrupted using different types of noise, including Gaussian noise and outliers. Gaussian noise is generated with varying parameters and a maximum standard deviation of 3 to increase dataset diversity. In addition, outliers are introduced by randomly selecting points that deviate significantly from the curve. To create a general dataset, we include curves with and without intersections, sampled uniformly or non-uniformly with varying numbers of points and different noise types and levels. The training set contains 396000 clean and noisy polygon curves. The curve data derived for this work will be made public and may be utilized in future work to train networks to learn various other curve features. A sample set of our synthetic data may be viewed here: https://anonymouscurves.github.io.

We additionally evaluate on CurveML (Raffo et al., 2024), a public dataset of Bezier and parametric curves with different noise types including Gaussian noise, outliers, and combined artifacts.

3.2 Training neural network

The network input consists of noisy ordered point sequences generated as described in the previous section. During training, the network learns to reconstruct a denoised polygonal curve approximating samples from the original smooth B-spline curve. The predicted output preserves the same number of sample points as the input. For applications involving unordered point sets, a preprocessing step may first be required to establish point ordering (Marin et al., 2022).

3.2.1 Investigating various network structures

The objective is to develop a denoising network capable of reconstructing smooth curves under varying noise distributions, sampling densities, and curve geometries. Since neighboring points on a curve exhibit strong spatial correlations, convolutional layers are employed for feature extraction (Komarichev et al., 2019).

The baseline architecture (Net_{fixed1}) consists of a shallow convolutional autoencoder with two convolutional layers, followed by batch normalization (Ioffe, 2015), and max pooling. To investigate the influence of architectural design choices, several variants were evaluated, including larger convolutional kernels (Net_{fixed2}), increased numbers of convolutional filters (Net_{fixed3}), and deeper network architectures (Net_{fixed4}). Table 1 summarize the quantitative and qualitative comparison of these configurations. Two metrics are utilized for evaluation: the normalized Hausdorff distance (HD), which measures the maximum nearest neighbor deviation, and the normalized Chamfer distance (CD), which measures the average bidirectional nearest neighbor distance. Normalization is done via dividing the error by the diagonal length of the bounding box that includes the target and predicted curves. Increasing the number of convolutional filters produced the largest improvement in reconstruction accuracy, while deeper architectures showed limited benefit and occasionally reduced preservation of fine geometric details. Furthermore, fixed-size architectures generalized poorly to curves with varying sampling densities and lengths. Zero-padding enabled variable-sized processing but introduced artifacts near curve endpoints.

Motivated by these observations, inception modules were incorporated between convolutional layers to improve multi-scale feature extraction. Unlike deeper sequential CNNs, inception modules simultaneously capture fine-scale local features and larger geometric structures, improving reconstruction quality without excessive feature loss. The resulting architecture produces smoother reconstructions and improved robustness across varying curve geometries and sampling densities.

Table 1. In this table, the average normalized Hausdorff (HD) and Chamfer (CD) distance between predicted and target curves is computed for 2000 examples for each network. Net_{fixed1} is a 2-layer CNN, Net_{fixed2} is a 2-layer CNN with increased kernel size from 3 to 5, Net_{fixed3} is a 2-layer CNN with increased number of kernels, Net_{fixed4} is a 3-layer CNN, and Net_{fixed5} is a 3-layer CNN with inception modules.

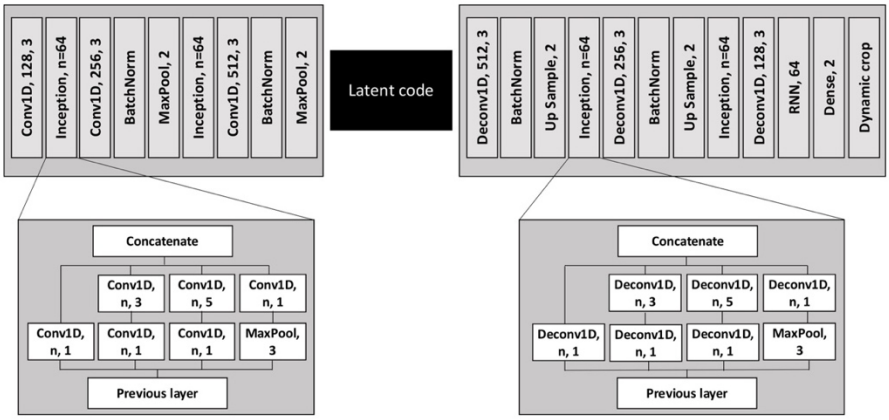
Network	Average HD	Average CD
Net_{fixed1}	0.0242	0.00011
Net_{fixed2}	0.0230	0.00012
Net_{fixed3}	0.0187	0.00007
Net_{fixed4}	0.0206	0.00008
Net_{fixed5}	0.0161	0.00005

Although the quantitative improvements are moderate, the inception-based architecture produces noticeably smoother reconstructions while preserving reconstruction accuracy, achieving a better balance between geometric fidelity and smoothness consistency.

3.2.2 The proposed network architecture

The architecture we propose for the denoising autoencoder is shown in Figure 1. The encoder includes three convolutional layers and two inception modules to enhance feature extraction. Each inception module consists of parallel convolutional layers with kernel sizes of 1, 3, and 5, applied simultaneously to the input feature maps and concatenated along the channel dimension. This design allows the network to capture curve features at multiple spatial scales, small kernels detect fine local point displacements, while larger kernels capture broader geometric context. A convolution with kernel size 1 is also included for channel-wise dimensionality reduction before the larger kernels. The decoder part serves as the counterpart to the encoder, responsible for reconstructing the original input data from the compressed representation provided by the encoder. To support variable curve lengths, the input layer is configured as *None*× 2. A recurrent layer (Medsker et al., 2001) is included in the decoder to process variable-length sequences, while a dynamic cropping layer ensures that the output size matches the input size. Padding is applied during training for batch processing. All convolutional layers use ReLU activations, except for the final layer, which uses Tanh to constrain the output to the interval [-1,1].

Figure 1. The architecture of the network.



To train the network, a gradient-based Adam optimizer (Kingma, 2014) with a learning rate of 0.001 is used. This network contains about 2 million parameters and is trained in 1147 epochs due to early stopping with a batch size of 32. Early stopping is used to avoid overfitting and enhance the generalization of our network. The experiments are conducted with Tensorflow, on a Linux server equipped with a GeForce RTX 4090 GPU. Training this network on a 396000 dataset took approximately one day. For evaluation, less than two seconds is required to provide a smooth polygon curve for a noisy input curve, irrespective of how many data points are provided as input to the network.

3.2.3 Loss function

The proposed loss function combines reconstruction accuracy with smoothness regularization. Mean Squared Error (MSE) is used to measure the difference between the target sampled points and the predicted polygonal curve. To encourage smooth geometric prediction, a Laplacian term based on the discrete second-order derivative, $\Delta_{Curve} = C_{i+1} - 2C_i + C_{i-1}$, is incorporated into the loss function. This regularization reduces abrupt changes in local curvature and improves tangential consistency between consecutive curve points. Although polygons are piecewise linear, the proposed smoothing term produces reconstructions that closely resemble smooth parametric curves. Figure 2 illustrates the effect of the Laplacian term on the behavior of tangent vectors and reconstruction smoothness. To further improve reconstruction quality, endpoint and chord length losses are included. The endpoint loss explicitly constrains the first and last curve points, while the chord length term preserves the overall curve length. The final loss function combines reconstruction, Laplacian smoothness, endpoint, and chord length losses to preserve both geometric fidelity and smoothness consistency.

The final loss function is:

$$L_{total} = L_{curve} + L_{laplacian} + L_{start} + L_{end} + 0.01L_{chord\_length}$$

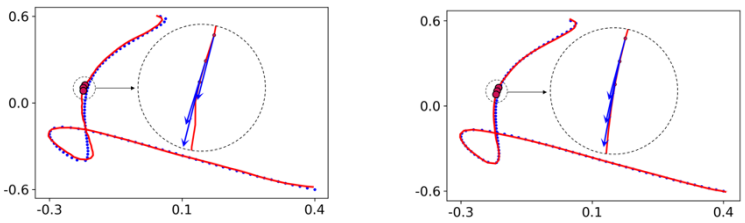
$$L_{curve} = \frac{1}{N}\sum_{i=1}^{N}(y_{true,i} - y_{pred,i})^2$$

$$L_{laplacian} = \frac{1}{N}\sum_{i=1}^{N}(\Delta y_{true,i} - \Delta y_{pred,i})$$

$$L_{start} = (y_{true,0} - y_{pred,0})^2,\ L_{end} = (y_{true,-1} - y_{pred,-1})^2$$

$$L_{chord_{length}} = |chord(y_{true}) - chord(y_{pred})|$$

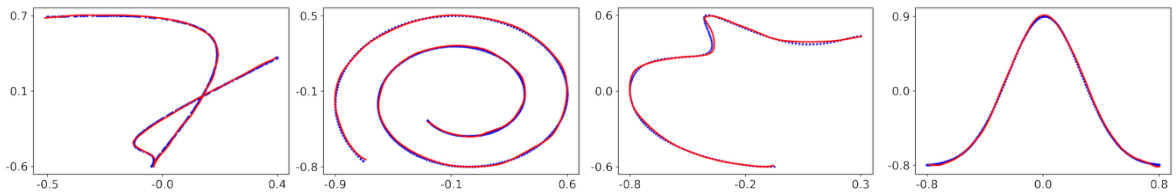
Figure 2. Effect of the Laplacian smoothing term on curve reconstruction. The left result is generated without Laplacian smoothing, while the right result includes it. The target curve is shown in blue, and the predicted curve is red.



4. RESULTS

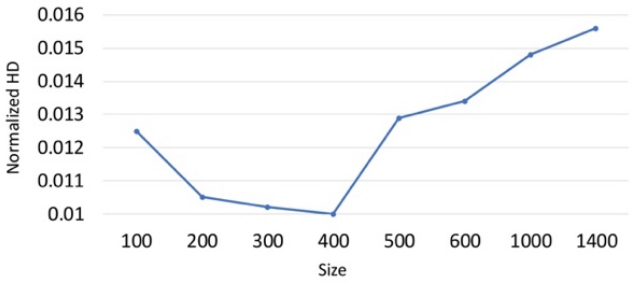
Figure 3 presents qualitative reconstruction results for several noisy curve configurations. The first example shows denoising results for curves corrupted by Gaussian noise included during training. The second example evaluates generalization on a CurveML sample, where performance slightly decreases due to differences in data distribution. The third example demonstrates robustness to combined Gaussian noise and outliers, although such noise combinations are not explicitly included during training. The final example shows that the network generalizes beyond the B-spline curves used during training and can reconstruct geometrically consistent non-parametric curves as well.

Figure 3. Some examples of denoising 2D curves. The target data curves are shown in blue, and the clean polygon curve reconstruction is depicted in red. The example on the right is a non-parametric curve: $f(t) = \frac{1}{3}e(-\frac{81}{4}(t-0.5)^2)$.



The training set contains curves ranging from 20×2 to 500×2. Although the network is trained only on curves up to this size, it can process inputs of different lengths without retraining, with reconstruction accuracy decreasing for very large curves. To evaluate robustness across different sampling densities, we test the network on 2000 noisy open cubic B-spline curves represented using 100-1400 points. Figure 4 illustrates the relationship between sampling density and reconstruction accuracy based on the normalized Hausdorff distance. Since most training examples contain 200-500 points, the network performs best within this range.

Figure 4. The relation between the size of the input curve and the performance of the proposed network



We compare the proposed approach with two classical smoothing techniques, namely the Savitzky–Golay algorithm (Savitzky and Golay, 1964) and averaging window filtering (Johnston et al., 1999), as well as the deep learning method ResCNN (Sun et al., 2020). See Section 2 for a discussion of recent curve reconstruction, point-set denoising, and 3D point cloud denoising methods, none of which directly address the present task. Parameters for the classical methods were empirically optimized for each noisy curve. While Savitzky–Golay and averaging window filtering perform reasonably well under Gaussian noise, they remain sensitive to outliers and complex noise configurations. Table 2 presents a quantitative evaluation on a test set of 2000 noisy curves using normalized Hausdorff and Chamfer distances and tangent behavior. To evaluate tangent smoothness, we compute the mean absolute change in tangent angle between consecutive points for each curve. The absolute difference between each method's value and the ground truth is averaged across the test set, yielding tangent deviation measure for each method, where lower values indicate closer match to the clean curve's directional behavior.

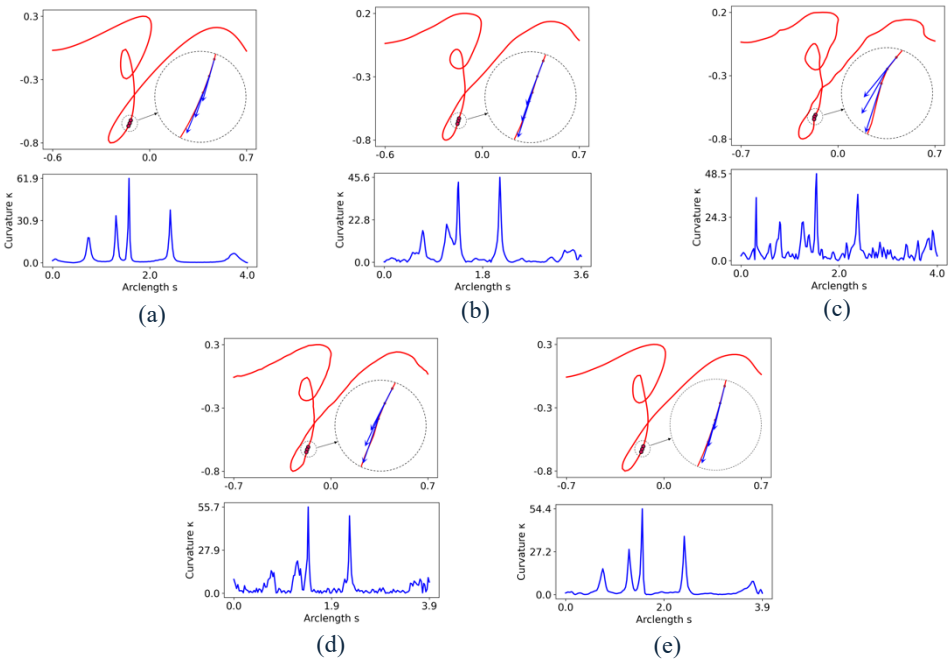
The proposed method achieves the lowest reconstruction error among all evaluated approaches (Table 2). Since distance-based metrics do not fully capture geometric smoothness, we additionally evaluate tangent behavior using the total variation of the tangent angle. The tangent variation of the proposed method deviates least from the ground truth among all the evaluated methods, indicating more preservation of directional smoothness. Figure 5 illustrates this qualitatively: while the averaging window and Savitzky-Golay outputs show oversmoothed tangent directions that loss fine geometric details, and ResCNN produces erratic directional changes, the proposed method's tangent vectors closely follow the target curve's directional behavior. These improvements are particularly important for applications requiring stable and visually coherent curve reconstruction.

Table 2. Comparison of denoising methods.

Method	Average HD	Average CD	Tangent deviation
Averaging window	0.0380	0.00020	0.0092
Savitzky-Golay	0.0259	0.00013	0.0089
ResCNN	0.0227	0.00015	0.0523
Proposed network	0.0212	0.00013	0.0066

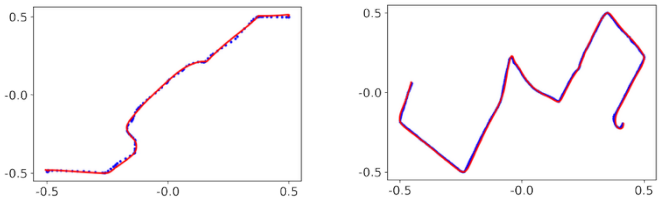
Beyond planar open curves, the proposed method generalizes to closed and 3D curves by operating on ordered point sequences. Through training on a diverse dataset containing multiple noise types and magnitudes, the network has the ability to generalize well, making it highly effective in real-world scenarios where noise characteristics can vary considerably. Utilizing this network in a preprocessing step to provide a clean input will improve the performance of any downstream applications.

Figure 5. The tangent vectors' directions on selected points of (a) smooth target curve, (b) the averaging window output, (c) the Savitzky-Golay output, (d) the ResCNN output, and (e) our proposed method.



Example application: GPS trajectories are often affected by sensor noise, which can hinder tasks such as map matching and road network reconstruction. We apply our method to real-world GPS trajectories from OpenStreetMap (http://www.openstreetmap.org) GPS traces, producing smooth reconstructions from noisy input for downstream analysis. Figure 6 compares some denoised results produced by our method (red) with raw noisy trajectories (blue).

Figure 6. Noisy trajectories (blue) and the corresponding denoised results obtained by our proposed method (red).



5. Conclusion

The objective of this work is to provide a fast and robust method for transforming noisy curve data into smooth polygonal representations, thereby enabling improved design, analysis, and visualization across diverse applications. The proposed network is flexible, capable of handling curves of variable length and subject to various noise distributions. To enhance generalization, we construct a large training dataset of open curves with different noise types and levels. We benchmark its performance against classical and deep learning–based methods, demonstrating that our approach consistently produces smoother and more accurate reconstructions. Since the reconstructed smooth polygons closely approximate smooth parametric curves, the method is particularly well suited to facilitating the reconstruction of smooth B-spline curve from corrupted data. However, the applicability of the proposed denoising network extends beyond parametric curves, with potential impact in areas where smooth polygonal representations are critical, including CAD/CAM, motion planning, medical imaging, GIS and cartography, and engineering analysis.

6. REFERENCES

Al-Zawi, S. et al., 2017. *Understanding of a convolutional neural network. Proceedings of the 2017 International Conference on Engineering and Technology (ICET)*, pp. 1-6.

Barzegar Khalilsaraei, S. et al., 2024. *InceptCurves: curve reconstruction using an inception network. The Visual Computer*, 40(7), pp. 4805-4815.

Cheng, S.-W. et al., 2003. *Curve reconstruction from noisy samples. Proceedings of the Nineteenth Annual Symposium on Computational Geometry*, pp. 302-311.

Cleveland, W. S., 1979. *Robust locally weighted regression and smoothing scatterplots. Journal of the American Statistical Association*, 74(368), pp. 829-836.

De Boor, C., 1972. On calculating with B-splines. *Journal of Approximation Theory*, 6(1), pp. 50-62.

Ebrahimi, A. and Loghmani, G. B., 2019. *B-spline curve fitting by diagonal approximation BFGS methods. Iranian Journal of Science and Technology, Transactions A: Scienc*e, 43(3), pp. 947-958.

Fan, C.-L. and Pang, M.-Y., 2009. *Reconstructing smooth curve from noise sampled data. Proceedings of The 2009 International Workshop on Information Security and Application (IWISA 2009)*. pp. 218.

Feiszli, M. and Jones, P. W., 2011. *Curve denoising by multiscale singularity detection and geometric shrinkage. Applied and Computational Harmonic Analysis,* 31(3), pp. 392-409.

Gao, J. et al., 2019. *Deepspline: Data-driven reconstruction of parametric curves and surfaces. arXiv preprint arXiv*:1901.03781.

Gálvez, A. and Iglesias, A., 2013. *Firefly algorithm for explicit B-spline curve fitting to data points. Mathematical Problems in Engineering*, 2013(1), p. 528215.

Ioffe, S. and Szegedy, C., 2015. *Batch normalization: Accelerating deep network training by reducing internal covariate shift. Proceedings of the International Conference on Machine Learning*, pp. 448-456.

Johnston, F. et al., 1999. *Some properties of a simple moving average when applied to forecasting a time series. Journal of the Operational Research Society*, 50(12), pp. 1267-1271.

Kingma, D. P., 2014. *Adam: A method for stochastic optimization. arXiv preprint arXiv*:1412.6980.

Komar, A. and Augsdörfer, U., 2023. *Swarmcurves: Evolutionary curve reconstruction. Proceedings of the International Symposium on Visual Computing*, pp. 343-354.

Komarichev, A. et al., 2019. *A-CNN: Annularly convolutional neural networks on point clouds. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7421-7430.

Laube, P. et al., 2018. *Deep learning parametrization for B-spline curve approximation. Proceedings of the 2018 International Conference on 3D Vision (3DV),* pp. 691-699.

Lin, S., & Feng, J., 2025. *A parametrization method for B-spline curve interpolation via supervised regression. Computers & Graphics*, 104360.

Lu, Z. and Zhong, B., 2015. *A rapid algorithm for curve denoising. Proceedings of the 2015 Chinese Automation Congress (CAC)*, pp. 717-721.

Luo, Shitong, and Wei Hu., 2021. *Score-based point cloud denoising. Proceedings of the IEEE/CVF international conference on computer vision*.

Marin, D. et al., 2022. *Sigdt: 2D curve reconstruction. Computer Graphics Forum*, 41, pp. 25-36.

Medsker, L. R. et al., 2001. *Recurrent neural networks. Design and Applications*, 5, pp. 64-67.

Morvan, M. et al., 2022. *Don't pay attention to the noise: Learning self-supervised representations of light curves with a denoising time series transformer. arXiv preprint arXiv*:2207.02777.

Raffo, A. et al., 2024. *CurveML: a benchmark for evaluating and training learning-based methods of classification, recognition, and fitting of plane curves. The Visual Computer*, 40(12), pp. 9017-9037.

Reghunath, Minu, et al., 2025. *Learning-based geometric framework for noise discernment and denoising of 2D point sets. Computers &* Graphics:104327.

Savitzky, A. and Golay, M. J., 1964. *Smoothing and differentiation of data by simplified least squares procedures. Analytical Chemistry*, 36(8), pp. 1627-1639.

Sun, W. et al., 2020. *A novel end-to-end 1D-ResCNN model to remove artifact from EEG signals. Neurocomputing*, 404, pp. 108-121.

Szegedy, C. et al., 2015. *Going deeper with convolutions. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1-9.

Vaswani, A. et al., 2017. *Attention is all you need. Advances in Neural Information Processing Systems*, 30.

Zou, Qiang, et al., 2025. *SplineGen: Approximating unorganized points through generative AI. Computer-Aided Design* 178: 103809.